

Start-Issue Prompt (copy-paste)

The best way to start a plan issue is the slash command `/solve-issue <number>` in Claude Code — it runs everything below automatically. This document is the **manual twin** for when you're not in Claude Code (e.g. claude.ai, a fresh agent, another tool): paste the block below as your first message and fill in the issue number.

Prerequisite for the automated checks: the assistant must have shell access with the `gh` CLI authenticated against `NCollection-Sys/ncollection-erp` and this repo checked out. Without that, it can still do Phases 4–6 if you paste the issue body yourself.

You are starting work on ONE GitHub issue of the NCollection ERP project (repo NCollection-Sys/ncollection-erp). Read CLAUDE.md in the repo root first for project context. Follow these phases IN ORDER and do not edit any file until I approve the plan in Phase 5.

Issue number: _____

PHASE 1 – IDENTIFY

If I didn't give an issue number above, ask me for it and nothing else.

PHASE 2 – AUTOMATED VERIFICATION (read-only). Never use gh search with [brackets]; fetch JSON and filter with --jq.

1. Open? `gh issue view <N> --repo NCollection-Sys/ncollection-erp --json state,title,body,labels` → state must be OPEN (if CLOSED, stop and tell me when/why).
2. Task ID: extract [P<phase>-T<nn>] from the title. If none, it's not a plan issue – skip checks 3–5 and confirm scope with me.
3. Deferred guard: label phase:9-marketplace → BLOCK (Phase 9 deferred until after Phase 10).
4. DEPENDENCY GATE (hard): parse the body line `“**Dependencies**<br>...”`. For each task ID, `gh issue list --state all --limit 300 --json number,title,state --jq '.[] | select(.title | startswith("[<ID>"]))'`
Convention: closed issue = completed task. Every dependency must be CLOSED. Any OPEN dependency → BLOCK and list each blocker (#num, ID, title, url).
5. Chronology (advisory): read the task's row in docs/markdown/DELIVERABLE_1_SYSTEM_DESIGN.md and its sprint in docs/markdown/SPRINT_SCHEDULE.md. Warn if it's scheduled later than still-open earlier-sprint tasks, but dependencies are the only hard block.
6. Duplicate guard: `gh pr list --state all --json number,title,state --jq '.[] | select(.title | contains("<ID>"))'` → warn on any existing PR.
7. Preflight: `git status clean; git checkout develop && git pull --ff-only; make ps` (offer make up if down).

PHASE 3 – REPORT + GATE

Show a ✓/x/△ table of all checks. All ✓ → continue. Any x → STOP and give me three options: (a) switch to the blocking dependency issue, (b) override and proceed at my explicit risk (note it in the PR), (c) abort. Never proceed past a x silently.

PHASE 4 – LOAD CONTEXT

Read: the task's full row in DELIVERABLE_1 (quote the acceptance criteria); the Standing Rules in docs/markdown/TASK_PROMPT_TEMPLATE.md; and the domain deep-dive doc (auth/licensing/security → ARCHITECTURE_SECURITY.md; provisioning/DB/infra → ARCHITECTURE_DATA_PLATFORM.md; demo porting → demo/README.md). Restate scope, acceptance criteria as a checklist, files in scope, and out-of-scope. Ask about anything ambiguous.

PHASE 5 – PLAN GATE

Give me a mini plan (steps, files, tests, risks). Wait for my explicit approval.

PHASE 6 – EXECUTE + SHIP

Branch feature/<N>-<task-id> off develop. Small verified commits. Before pushing run: `flake8 custom_addons/ ; python3 scripts/ci/architecture_guard.py --base origin/develop ;` (and `cd demo && npx tsc --noEmit` if demo/ changed). Open a PR to develop titled "[<ID>] <name>" with test evidence and "Closes #<N>". Watch the 4 CI checks. Do NOT merge your own PR. Finish by listing the issues this one unblocks as next candidates.

GUARDRAILS: one issue per chat; never touch other issues; no scope creep (propose follow-up issues instead); no secrets in git. Odoo 19: routes are `type='jsonrpc'`; `res.users` groups field is `group_ids`; views use <list> not <tree>, no `attrs=`; the X-Odoo-Database header suppresses the session cookie (only use it on public session-less calls).

Both this doc and the `/solve-issue` command are kept in sync; the command is the source of truth. See also `CLAUDE.md` (project context) and `docs/markdown/TASK_PROMPT_TEMPLATE.md` (the canonical Standing Rules).